Automatic Addison
Build the Future

Easy Cancel'ations
Whatever Your Course, We Can Rent You The Right Property.
University Living

The Ultimate Guide to Real-Time Lane Detection Using OpenCV



Alternative for SMEs - Free Forever
py Customers. 2 Weeks Implementation. Accounts Integration
stranzact.com/free_erp

In this tutorial, we will go through the entire process, step by step, of how to detect lanes on a road in real time using the OpenCV computer vision library and Python. By the end of this tutorial, you will know how to build (from scratch) an application that can automatically detect lanes in a video stream from a front-facing camera mounted on a car. You'll be able to generate this video below.



Our goal is to create a program that can read a video stream and output an annotated video that shows the following:

1. The current lane
2. The radius of curvature of the lane
3. The position of the vehicle relative to the middle of the lane

In a future post, we will use #3 to control the steering angle of a self-driving car in the CARLA autonomous driving simulator.

Table of Contents

Real-World Applications
Prerequisites
Helpful Tip
Find Some Videos and an Image
Installation and Setup
Python Code for Detection of Lane Lines in an Image
Isolate Pixels That Could Represent Lane Lines
Thresholding Steps
Apply Perspective Transformation to Get a Bird's Eye View
Why We Need to Do Perspective Transformation
How Perspective Transformation Works
Identify Lane Line Pixels
Set Sliding Windows for White Pixel Detection
Fill in the Lane Line
Overlay Lane Lines on Original Image

Alternative for SMEs - Free Forever
by Customers, 2 Weeks Implementation, Accounts Integration
transact.com/free_erp

Real-World Applications

- Self-Driving Cars

Prerequisites

- Python 3.7 or higher with OpenCV installed

Helpful Tip

As you work through this tutorial, focus on the end goals I listed in the beginning. Don't get bogged down in trying to understand every last detail of the math and the OpenCV operations we'll use in our code (e.g. bitwise AND, Sobel edge detection algorithm etc.). Trust the developers at Intel who manage the OpenCV computer vision package.

We are trying to build products not publish research papers. Focus on the inputs, the outputs, and what the algorithm is supposed to do at a high level.

NFTs and Mystery Boxes

Get a working lane detection application up and running; and, at some later date when you want to add more complexity to your project or write a research paper, you can dive deeper under the hood to understand all the details.

Trying to understand every last detail is like trying to build your own database from scratch in order to start a website or taking a course on internal combustion engines to learn how to drive a car.

Let's get started!

Find Some Videos and an Image

The first thing we need to do is find some videos and an image to serve as our test cases.

We want to download videos and an image that show a road with lanes from the perspective of a person driving a car.

Alternative for SMEs - Free Forever
py Customers, 2 Weeks Implementation, Accounts Integration
stranzact.com/free_erp

NFTs and Mystery Boxes

I found some good candidates on Pixabay.com. Type “driving” or “lanes” in the video search on that website.

Here is an example of what a frame from one of your videos should look like. This frame is 600 pixels in width and 338 pixels in height:



Installation and Setup

We now need to make sure we have all the software packages installed. Check to see if you have **OpenCV** installed on your machine. If you are using Anaconda, you can type:

```
conda install -c conda-forge opencv
```

Alternatively, you can type:

```
pip install opencv-python
```

Make sure you have **NumPy** installed, a scientific computing library for Python.

If you’re using Anaconda, you can type:

```
conda install numpy
```

Alternatively, you can type:

Alternative for SMEs - Free Forever
py Customers, 2 Weeks Implementation, Accounts Integration
stranzact.com/free_erp

For Anaconda users:

```
conda install -c conda-forge matplotlib
```

Otherwise, you can install like this:

```
pip install matplotlib
```

Python Code for Detection of Lane Lines in an Image

Before, we get started, I'll share with you the full code you need to perform lane detection in an image. The two programs below are all you need to detect lane lines in an image.

NFTs and Mystery Boxes

You need to make sure that you save both programs below, **edge_detection.py** and **lane.py** in the same directory as the image.

edge_detection.py will be a collection of methods that helps isolate lane line edges and lane lines.

lane.py is where we will implement a Lane class that represents a lane on a road or highway.

Don't be scared at how long the code appears. I always include a lot of comments in my code since I have the tendency to forget why I did what I did. I always want to be able to revisit my code at a later date and have a clear understanding what I did and why:

NFTs and Mystery Boxes

Here is **edge_detection.py**. Don't worry, I'll explain the code later in this post.

```
1 import cv2 # Import the OpenCV library to enable computer vision
```

Alternative for SMEs - Free Forever
py Customers, 2 Weeks Implementation, Accounts Integration
stranzact.com/free_erp

```
    """
    Return a 2D binary array (mask) in which all pixels are either 0 or 1

    :param array: NumPy 2D array that we want to convert to binary values
    :param thresh: Values used for thresholding (inclusive)
    :param value: Output value when between the supplied threshold
    :return: Binary 2D array...
             number of rows x number of columns =
             number of pixels from top to bottom x number of pixels from
               left to right
    """
    if value == 0:
      # Create an array of ones with the same shape and type as
      # the input 2D array.
      binary = np.ones_like(array)

    else:
      # Creates an array of zeros with the same shape and type as
      # the input 2D array.
      binary = np.zeros_like(array)
      value = 1

    # If value == 0, make all values in binary equal to 0 if the
    # corresponding value in the input array is between the threshold
    # (inclusive). Otherwise, the value remains as 1. Therefore, the pixels
    # with the high Sobel derivative values (i.e. sharp pixel intensity
    # discontinuities) will have 0 in the corresponding cell of binary.
    binary[(array >= thresh[0]) & (array <= thresh[1])] = value

    return binary

  def blur_gaussian(channel, ksize=3):
    """
    Implementation for Gaussian blur to reduce noise and detail in the image

    :param image: 2D or 3D array to be blurred
    :param ksize: Size of the small matrix (i.e. kernel) used to blur
                  i.e. number of rows and number of columns
    :return: Blurred 2D image
    """
    return cv2.GaussianBlur(channel, (ksize, ksize), 0)

  def mag_thresh(image, sobel_kernel=3, thresh=(0, 255)):
    """
    Implementation of Sobel edge detection

    :param image: 2D or 3D array to be blurred
    :param sobel_kernel: Size of the small matrix (i.e. kernel)
                         i.e. number of rows and columns
    :return: Binary (black and white) 2D mask image
    """
    # Get the magnitude of the edges that are vertically aligned on the image
    sobelx = np.absolute(sobel(image, orient='x', sobel_kernel=sobel_kernel))

    # Get the magnitude of the edges that are horizontally aligned on the image
    sobely = np.absolute(sobel(image, orient='y', sobel_kernel=sobel_kernel))

    # Find areas of the image that have the strongest pixel intensity changes
    # in both the x and y directions. These have the strongest gradients and
    # represent the strongest edges in the image (i.e. potential lane lines)
    # mag is a 2D array .. number of rows x number of columns = number of pixels
    # from top to bottom x number of pixels from left to right
    mag = np.sqrt(sobelx ** 2 + sobely ** 2)

    # Return a 2D array that contains 0s and 1s
    return binary_array(mag, thresh)

  def sobel(img_channel, orient='x', sobel_kernel=3):
    """
    Find edges that are aligned vertically and horizontally on the image

    :param img_channel: Channel from an image
    :param orient: Across which axis of the image are we detecting edges?
    :sobel_kernel: No. of rows and columns of the kernel (i.e. 3x3 small matrix)
    :return: Image with Sobel edge detection applied
    """
```

Alternative for SMEs - Free Forever
py Customers, 2 Weeks Implementation, Accounts Integration
stranzact.com/free_erp

```
    # Will detect differences in pixel intensities going from
    # top to bottom on the image (i.e. edges that are horizontally aligned)
    sobel = cv2.Sobel(img_channel, cv2.CV_64F, 0, 1, sobel_kernel)

  return sobel

def threshold(channel, thresh=(128,255), thresh_type=cv2.THRESH_BINARY):
  """
  Apply a threshold to the input channel

  :param channel: 2D array of the channel data of an image/video frame
  :param thresh: 2D tuple of min and max threshold values
  :param thresh_type: The technique of the threshold to apply
  :return: Two outputs are returned:
            ret: Threshold that was used
            thresholded_image: 2D thresholded data.
  """
  # If pixel intensity is greater than thresh[0], make that value
  # white (255), else set it to black (0)
  return cv2.threshold(channel, thresh[0], thresh[1], thresh_type)
```

Here is **lane.py**.

```
import cv2 # Import the OpenCV library to enable computer vision
import numpy as np # Import the NumPy scientific computing library
import edge_detection as edge # Handles the detection of lane lines
import matplotlib.pyplot as plt # Used for plotting and error checking

# Author: Addison Sears-Collins
# https://automaticaddison.com
# Description: Implementation of the Lane class

filename = 'original_lane_detection_5.jpg'

class Lane:
  """
  Represents a lane on a road.
  """
  def __init__(self, orig_frame):
    """
      Default constructor

    :param orig_frame: Original camera image (i.e. frame)
    """
    self.orig_frame = orig_frame

    # This will hold an image with the lane lines
    self.lane_line_markings = None

    # This will hold the image after perspective transformation
    self.warped_frame = None
    self.transformation_matrix = None
    self.inv_transformation_matrix = None

    # (Width, Height) of the original video frame (or image)
    self.orig_image_size = self.orig_frame.shape[::-1][1:]

    width = self.orig_image_size[0]
    height = self.orig_image_size[1]
    self.width = width
    self.height = height

    # Four corners of the trapezoid-shaped region of interest
    # You need to find these corners manually.
    self.roi_points = np.float32([
      (274,184), # Top-left corner
      (0, 337), # Bottom-left corner
      (575,337), # Bottom-right corner
      (371,184) # Top-right corner
    ])

    # The desired corner locations  of the region of interest
    # after we perform perspective transformation.
    # Assume image width of 600, padding == 150.
```

Alternative for SMEs - Free Forever
py Customers, 2 Weeks Implementation, Accounts Integration
stranzact.com/free_erp

```
    ])

    # Histogram that shows the white pixel peaks for lane line detection
    self.histogram = None

    # Sliding window parameters
    self.no_of_windows = 10
    self.margin = int((1/12) * width)  # Window width is +/- margin
    self.minpix = int((1/24) * width)  # Min no. of pixels to recenter window

    # Best fit polynomial lines for left line and right line of the lane
    self.left_fit = None
    self.right_fit = None
    self.left_lane_inds = None
    self.right_lane_inds = None
    self.ploty = None
    self.left_fitx = None
    self.right_fitx = None
    self.leftx = None
    self.rightx = None
    self.lefty = None
    self.righty = None

    # Pixel parameters for x and y dimensions
    self.YM_PER_PIX = 10.0 / 1000 # meters per pixel in y dimension
    self.XM_PER_PIX = 3.7 / 781 # meters per pixel in x dimension

    # Radii of curvature and offset
    self.left_curvem = None
    self.right_curvem = None
    self.center_offset = None

  def calculate_car_position(self, print_to_terminal=False):
    """
    Calculate the position of the car relative to the center

    :param: print_to_terminal Display data to console if True
    :return: Offset from the center of the lane
    """
    # Assume the camera is centered in the image.
    # Get position of car in centimeters
    car_location = self.orig_frame.shape[1] / 2

    # Fine the x coordinate of the lane line bottom
    height = self.orig_frame.shape[0]
    bottom_left = self.left_fit[0]*height**2 + self.left_fit[
      1]*height + self.left_fit[2]
    bottom_right = self.right_fit[0]*height**2 + self.right_fit[
      1]*height + self.right_fit[2]

    center_lane = (bottom_right - bottom_left)/2 + bottom_left
    center_offset = (np.abs(car_location) - np.abs(
      center_lane)) * self.XM_PER_PIX * 100

    if print_to_terminal == True:
      print(str(center_offset) + 'cm')

    self.center_offset = center_offset

    return center_offset

  def calculate_curvature(self, print_to_terminal=False):
    """
    Calculate the road curvature in meters.

    :param: print_to_terminal Display data to console if True
    :return: Radii of curvature
    """
    # Set the y-value where we want to calculate the road curvature.
    # Select the maximum y-value, which is the bottom of the frame.
    y_eval = np.max(self.ploty)

    # Fit polynomial curves to the real world environment
    left_fit_cr = np.polyfit(self.lefty * self.YM_PER_PIX, self.leftx * (
      self.XM_PER_PIX), 2)
    right_fit_cr = np.polyfit(self.righty * self.YM_PER_PIX, self.rightx * (
```

Alternative for SMEs - Free Forever
by Customers, 2 Weeks Implementation, Accounts Integration
stranzact.com/free_erp

```
                    1])**2)**1.5) / np.absolute(2*right_fit_cr[0])

      # Display on terminal window
      if print_to_terminal == True:
        print(left_curvem, 'm', right_curvem, 'm')

      self.left_curvem = left_curvem
      self.right_curvem = right_curvem

      return left_curvem, right_curvem

    def calculate_histogram(self,frame=None,plot=True):
      """
      Calculate the image histogram to find peaks in white pixel count

      :param frame: The warped image
      :param plot: Create a plot if True
      """
      if frame is None:
        frame = self.warped_frame

      # Generate the histogram
      self.histogram = np.sum(frame[int(
                frame.shape[0]/2):,:], axis=0)

      if plot == True:

        # Draw both the image and the histogram
        figure, (ax1, ax2) = plt.subplots(2,1) # 2 row, 1 columns
        figure.set_size_inches(10, 5)
        ax1.imshow(frame, cmap='gray')
        ax1.set_title("Warped Binary Frame")
        ax2.plot(self.histogram)
        ax2.set_title("Histogram Peaks")
        plt.show()

      return self.histogram

    def display_curvature_offset(self, frame=None, plot=False):
      """
      Display curvature and offset statistics on the image

      :param: plot Display the plot if True
      :return: Image with lane lines and curvature
      """
      image_copy = None
      if frame is None:
        image_copy = self.orig_frame.copy()
      else:
        image_copy = frame

      cv2.putText(image_copy,'Curve Radius: '+str((
        self.left_curvem+self.right_curvem)/2)[:7]+' m', (int((
        5/600)*self.width), int((
        20/338)*self.height)), cv2.FONT_HERSHEY_SIMPLEX, (float((
        0.5/600)*self.width)),(
        255,255,255),2,cv2.LINE_AA)
      cv2.putText(image_copy,'Center Offset: '+str(
        self.center_offset)[:7]+' cm', (int((
        5/600)*self.width), int((
        40/338)*self.height)), cv2.FONT_HERSHEY_SIMPLEX, (float((
        0.5/600)*self.width)),(
        255,255,255),2,cv2.LINE_AA)

      if plot==True:
        cv2.imshow("Image with Curvature and Offset", image_copy)

      return image_copy

    def get_lane_line_previous_window(self, left_fit, right_fit, plot=False):
      """
      Use the lane line from the previous sliding window to get the parameters
      for the polynomial line for filling in the lane line
      :param: left_fit Polynomial function of the left lane line
      :param: right_fit Polynomial function of the right lane line
      :param: plot To display an image or not
      """
```

Alternative for SMEs - Free Forever

by Customers, 2 Weeks Implementation, Accounts Integration

stranzact.com/free_erp

```
    nonzeroy = np.array(nonzero[0])
    nonzerox = np.array(nonzero[1])

    # Store left and right lane pixel indices
    left_lane_inds = ((nonzerox > (left_fit[0]*(
      nonzeroy**2) + left_fit[1]*nonzeroy + left_fit[2] - margin)) & (
      nonzerox < (left_fit[0]*(
      nonzeroy**2) + left_fit[1]*nonzeroy + left_fit[2] + margin)))
    right_lane_inds = ((nonzerox > (right_fit[0]*(
      nonzeroy**2) + right_fit[1]*nonzeroy + right_fit[2] - margin)) & (
      nonzerox < (right_fit[0]*(
      nonzeroy**2) + right_fit[1]*nonzeroy + right_fit[2] + margin)))
    self.left_lane_inds = left_lane_inds
    self.right_lane_inds = right_lane_inds

    # Get the left and right lane line pixel locations
    leftx = nonzerox[left_lane_inds]
    lefty = nonzeroy[left_lane_inds]
    rightx = nonzerox[right_lane_inds]
    righty = nonzeroy[right_lane_inds]

    self.leftx = leftx
    self.rightx = rightx
    self.lefty = lefty
    self.righty = righty

    # Fit a second order polynomial curve to each lane line
    left_fit = np.polyfit(lefty, leftx, 2)
    right_fit = np.polyfit(righty, rightx, 2)
    self.left_fit = left_fit
    self.right_fit = right_fit

    # Create the x and y values to plot on the image
    ploty = np.linspace(
      0, self.warped_frame.shape[0]-1, self.warped_frame.shape[0])
    left_fitx = left_fit[0]*ploty**2 + left_fit[1]*ploty + left_fit[2]
    right_fitx = right_fit[0]*ploty**2 + right_fit[1]*ploty + right_fit[2]
    self.ploty = ploty
    self.left_fitx = left_fitx
    self.right_fitx = right_fitx

    if plot==True:

      # Generate images to draw on
      out_img = np.dstack((self.warped_frame, self.warped_frame, (
                           self.warped_frame)))*255
      window_img = np.zeros_like(out_img)

      # Add color to the left and right line pixels
      out_img[nonzeroy[left_lane_inds], nonzerox[left_lane_inds]] = [255, 0, 0]
      out_img[nonzeroy[right_lane_inds], nonzerox[right_lane_inds]] = [
                                                                  0, 0, 255]
      # Create a polygon to show the search window area, and recast
      # the x and y points into a usable format for cv2.fillPoly()
      margin = self.margin
      left_line_window1 = np.array([np.transpose(np.vstack([
                                    left_fitx-margin, ploty]))])
      left_line_window2 = np.array([np.flipud(np.transpose(np.vstack([
                                    left_fitx+margin, ploty])))])
      left_line_pts = np.hstack((left_line_window1, left_line_window2))
      right_line_window1 = np.array([np.transpose(np.vstack([
                                     right_fitx-margin, ploty]))])
      right_line_window2 = np.array([np.flipud(np.transpose(np.vstack([
                                     right_fitx+margin, ploty])))])
      right_line_pts = np.hstack((right_line_window1, right_line_window2))

      # Draw the lane onto the warped blank image
      cv2.fillPoly(window_img, np.int_([left_line_pts]), (0,255, 0))
      cv2.fillPoly(window_img, np.int_([right_line_pts]), (0,255, 0))
      result = cv2.addWeighted(out_img, 1, window_img, 0.3, 0)

      # Plot the figures
      figure, (ax1, ax2, ax3) = plt.subplots(3,1) # 3 rows, 1 column
      figure.set_size_inches(10, 10)
      figure.tight_layout(pad=3.0)
      ax1.imshow(cv2.cvtColor(self.orig_frame, cv2.COLOR_BGR2RGB))
```

Alternative for SMEs - Free Forever

by Customers, 2 Weeks Implementation, Accounts Integration

stranzact.com/free_erp

```
      plt.show()

  def get_lane_line_indices_sliding_windows(self, plot=False):
    """
    Get the indices of the lane line pixels using the
    sliding windows technique.

    :param: plot Show plot or not
    :return: Best fit lines for the left and right lines of the current lane
    """
    # Sliding window width is +/- margin
    margin = self.margin

    frame_sliding_window = self.warped_frame.copy()

    # Set the height of the sliding windows
    window_height = np.int(self.warped_frame.shape[0]/self.no_of_windows)

    # Find the x and y coordinates of all the nonzero
    # (i.e. white) pixels in the frame.
    nonzero = self.warped_frame.nonzero()
    nonzeroy = np.array(nonzero[0])
    nonzerox = np.array(nonzero[1])

    # Store the pixel indices for the left and right lane lines
    left_lane_inds = []
    right_lane_inds = []

    # Current positions for pixel indices for each window,
    # which we will continue to update
    leftx_base, rightx_base = self.histogram_peak()
    leftx_current = leftx_base
    rightx_current = rightx_base

    # Go through one window at a time
    no_of_windows = self.no_of_windows

    for window in range(no_of_windows):

      # Identify window boundaries in x and y (and right and left)
      win_y_low = self.warped_frame.shape[0] - (window + 1) * window_height
      win_y_high = self.warped_frame.shape[0] - window * window_height
      win_xleft_low = leftx_current - margin
      win_xleft_high = leftx_current + margin
      win_xright_low = rightx_current - margin
      win_xright_high = rightx_current + margin
      cv2.rectangle(frame_sliding_window,(win_xleft_low,win_y_low),(
        win_xleft_high,win_y_high), (255,255,255), 2)
      cv2.rectangle(frame_sliding_window,(win_xright_low,win_y_low),(
        win_xright_high,win_y_high), (255,255,255), 2)

      # Identify the nonzero pixels in x and y within the window
      good_left_inds = ((nonzeroy >= win_y_low) & (nonzeroy < win_y_high) &
                        (nonzerox >= win_xleft_low) & (
                         nonzerox < win_xleft_high)).nonzero()[0]
      good_right_inds = ((nonzeroy >= win_y_low) & (nonzeroy < win_y_high) &
                         (nonzerox >= win_xright_low) & (
                          nonzerox < win_xright_high)).nonzero()[0]

      # Append these indices to the lists
      left_lane_inds.append(good_left_inds)
      right_lane_inds.append(good_right_inds)

      # If you found > minpix pixels, recenter next window on mean position
      minpix = self.minpix
      if len(good_left_inds) > minpix:
        leftx_current = np.int(np.mean(nonzerox[good_left_inds]))
      if len(good_right_inds) > minpix:
        rightx_current = np.int(np.mean(nonzerox[good_right_inds]))

    # Concatenate the arrays of indices
    left_lane_inds = np.concatenate(left_lane_inds)
    right_lane_inds = np.concatenate(right_lane_inds)

    # Extract the pixel coordinates for the left and right lane lines
    leftx = nonzerox[left_lane_inds]
```

Alternative for SMEs - Free Forever
by Customers, 2 Weeks Implementation, Accounts Integration
stranzact.com/free_erp

```
    right_fit = np.polyfit(righty, rightx, 2)

    self.left_fit = left_fit
    self.right_fit = right_fit

    if plot==True:

      # Create the x and y values to plot on the image
      ploty = np.linspace(
        0, frame_sliding_window.shape[0]-1, frame_sliding_window.shape[0])
      left_fitx = left_fit[0]*ploty**2 + left_fit[1]*ploty + left_fit[2]
      right_fitx = right_fit[0]*ploty**2 + right_fit[1]*ploty + right_fit[2]

      # Generate an image to visualize the result
      out_img = np.dstack((
        frame_sliding_window, frame_sliding_window, (
        frame_sliding_window))) * 255

      # Add color to the left line pixels and right line pixels
      out_img[nonzeroy[left_lane_inds], nonzerox[left_lane_inds]] = [255, 0, 0]
      out_img[nonzeroy[right_lane_inds], nonzerox[right_lane_inds]] = [
        0, 0, 255]

      # Plot the figure with the sliding windows
      figure, (ax1, ax2, ax3) = plt.subplots(3,1) # 3 rows, 1 column
      figure.set_size_inches(10, 10)
      figure.tight_layout(pad=3.0)
      ax1.imshow(cv2.cvtColor(self.orig_frame, cv2.COLOR_BGR2RGB))
      ax2.imshow(frame_sliding_window, cmap='gray')
      ax3.imshow(out_img)
      ax3.plot(left_fitx, ploty, color='yellow')
      ax3.plot(right_fitx, ploty, color='yellow')
      ax1.set_title("Original Frame")
      ax2.set_title("Warped Frame with Sliding Windows")
      ax3.set_title("Detected Lane Lines with Sliding Windows")
      plt.show()

    return self.left_fit, self.right_fit

  def get_line_markings(self, frame=None):
    """
    Isolates lane lines.

      :param frame: The camera frame that contains the lanes we want to detect
    :return: Binary (i.e. black and white) image containing the lane lines.
    """
    if frame is None:
      frame = self.orig_frame

    # Convert the video frame from BGR (blue, green, red)
    # color space to HLS (hue, saturation, lightness).
    hls = cv2.cvtColor(frame, cv2.COLOR_BGR2HLS)

    ################### Isolate possible lane line edges ######################

    # Perform Sobel edge detection on the L (lightness) channel of
    # the image to detect sharp discontinuities in the pixel intensities
    # along the x and y axis of the video frame.
    # sxbinary is a matrix full of 0s (black) and 255 (white) intensity values
    # Relatively light pixels get made white. Dark pixels get made black.
    _, sxbinary = edge.threshold(hls[:, :, 1], thresh=(120, 255))
    sxbinary = edge.blur_gaussian(sxbinary, ksize=3) # Reduce noise

    # 1s will be in the cells with the highest Sobel derivative values
    # (i.e. strongest lane line edges)
    sxbinary = edge.mag_thresh(sxbinary, sobel_kernel=3, thresh=(110, 255))

    ######################## Isolate possible lane lines ######################

    # Perform binary thresholding on the S (saturation) channel
    # of the video frame. A high saturation value means the hue color is pure.
    # We expect lane lines to be nice, pure colors (i.e. solid white, yellow)
    # and have high saturation channel values.
    # s_binary is matrix full of 0s (black) and 255 (white) intensity values
    # White in the regions with the purest hue colors (e.g. >80...play with
    # this value for best results).
```

Alternative for SMEs - Free Forever
by Customers, 2 Weeks Implementation, Accounts Integration
tranzact.com/free_erp

```
    # Remember, pure white is bgr(255, 255, 255).
    # Pure yellow is bgr(0, 255, 255). Both have high red channel values.
    _, r_thresh = edge.threshold(frame[:, :, 2], thresh=(120, 255))

    # Lane lines should be pure in color and have high red channel values
    # Bitwise AND operation to reduce noise and black-out any pixels that
    # don't appear to be nice, pure, solid colors (like white or yellow lane
    # lines.)
    rs_binary = cv2.bitwise_and(s_binary, r_thresh)

    ### Combine the possible lane lines with the possible lane line edges #####
    # If you show rs_binary visually, you'll see that it is not that different
    # from this return value. The edges of lane lines are thin lines of pixels.
    self.lane_line_markings = cv2.bitwise_or(rs_binary, sxbinary.astype(
                              np.uint8))
    return self.lane_line_markings

  def histogram_peak(self):
    """
    Get the left and right peak of the histogram

    Return the x coordinate of the left histogram peak and the right histogram
    peak.
    """
    midpoint = np.int(self.histogram.shape[0]/2)
    leftx_base = np.argmax(self.histogram[:midpoint])
    rightx_base = np.argmax(self.histogram[midpoint:]) + midpoint

    # (x coordinate of left peak, x coordinate of right peak)
    return leftx_base, rightx_base

  def overlay_lane_lines(self, plot=False):
    """
    Overlay lane lines on the original frame
    :param: Plot the lane lines if True
    :return: Lane with overlay
    """
    # Generate an image to draw the lane lines on
    warp_zero = np.zeros_like(self.warped_frame).astype(np.uint8)
    color_warp = np.dstack((warp_zero, warp_zero, warp_zero))

    # Recast the x and y points into usable format for cv2.fillPoly()
    pts_left = np.array([np.transpose(np.vstack([
                         self.left_fitx, self.ploty]))])
    pts_right = np.array([np.flipud(np.transpose(np.vstack([
                          self.right_fitx, self.ploty])))])
    pts = np.hstack((pts_left, pts_right))

    # Draw lane on the warped blank image
    cv2.fillPoly(color_warp, np.int_([pts]), (0,255, 0))

    # Warp the blank back to original image space using inverse perspective
    # matrix (Minv)
    newwarp = cv2.warpPerspective(color_warp, self.inv_transformation_matrix, (
                                  self.orig_frame.shape[
                                  1], self.orig_frame.shape[0]))

    # Combine the result with the original image
    result = cv2.addWeighted(self.orig_frame, 1, newwarp, 0.3, 0)

    if plot==True:

      # Plot the figures
      figure, (ax1, ax2) = plt.subplots(2,1) # 2 rows, 1 column
      figure.set_size_inches(10, 10)
      figure.tight_layout(pad=3.0)
      ax1.imshow(cv2.cvtColor(self.orig_frame, cv2.COLOR_BGR2RGB))
      ax2.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
      ax1.set_title("Original Frame")
      ax2.set_title("Original Frame With Lane Overlay")
      plt.show()

    return result

  def perspective_transform(self, frame=None, plot=False):
    """
```

Alternative for SMEs - Free Forever
by Customers, 2 Weeks Implementation, Accounts Integration
stranzact.com/free_erp

```
557
558       # Calculate the transformation matrix
559       self.transformation_matrix = cv2.getPerspectiveTransform(
560         self.roi_points, self.desired_roi_points)
561
562       # Calculate the inverse transformation matrix
563       self.inv_transformation_matrix = cv2.getPerspectiveTransform(
564         self.desired_roi_points, self.roi_points)
565
566       # Perform the transform using the transformation matrix
567       self.warped_frame = cv2.warpPerspective(
568         frame, self.transformation_matrix, self.orig_image_size, flags=(
569        cv2.INTER_LINEAR))
570
571       # Convert image to binary
572       (thresh, binary_warped) = cv2.threshold(
573         self.warped_frame, 127, 255, cv2.THRESH_BINARY)
574       self.warped_frame = binary_warped
575
576       # Display the perspective transformed (i.e. warped) frame
577       if plot == True:
578         warped_copy = self.warped_frame.copy()
579         warped_plot = cv2.polylines(warped_copy, np.int32([
580                       self.desired_roi_points]), True, (147,20,255), 3)
581
582         # Display the image
583         while(1):
584           cv2.imshow('Warped Image', warped_plot)
585
586           # Press any key to stop
587           if cv2.waitKey(0):
588             break
589
590         cv2.destroyAllWindows()
591
592       return self.warped_frame
593
594     def plot_roi(self, frame=None, plot=False):
595       """
596       Plot the region of interest on an image.
597       :param: frame The current image frame
598       :param: plot Plot the roi image if True
599       """
600       if plot == False:
601         return
602
603       if frame is None:
604         frame = self.orig_frame.copy()
605
606       # Overlay trapezoid on the frame
607       this_image = cv2.polylines(frame, np.int32([
608         self.roi_points]), True, (147,20,255), 3)
609
610       # Display the image
611       while(1):
612         cv2.imshow('ROI Image', this_image)
613
614         # Press any key to stop
615         if cv2.waitKey(0):
616           break
617
618       cv2.destroyAllWindows()
619
620   def main():
621
622     # Load a frame (or image)
623     original_frame = cv2.imread(filename)
624
625     # Create a Lane object
626     lane_obj = Lane(orig_frame=original_frame)
627
628     # Perform thresholding to isolate lane lines
629     lane_line_markings = lane_obj.get_line_markings()
630
631     # Plot the region of interest on the image
632     lane_obj.plot_roi(plot=False)
```

Alternative for SMEs - Free Forever
by Customers, 2 Weeks Implementation, Accounts Integration
stranzact.com/free_erp

```
    histogram = lane_obj.calculate_histogram(plot=False)

    # Find lane line pixels using the sliding window method
    left_fit, right_fit = lane_obj.get_lane_line_indices_sliding_windows(
      plot=False)

    # Fill in the lane line
    lane_obj.get_lane_line_previous_window(left_fit, right_fit, plot=False)

    # Overlay lines on the original frame
    frame_with_lane_lines = lane_obj.overlay_lane_lines(plot=False)

    # Calculate lane line curvature (left and right lane lines)
    lane_obj.calculate_curvature(print_to_terminal=False)

    # Calculate center offset
    lane_obj.calculate_car_position(print_to_terminal=False)

    # Display curvature and center offset on image
    frame_with_lane_lines2 = lane_obj.display_curvature_offset(
      frame=frame_with_lane_lines, plot=True)

    # Create the output file name by removing the '.jpg' part
    size = len(filename)
    new_filename = filename[:size - 4]
    new_filename = new_filename + '_thresholded.jpg'

    # Save the new image in the working directory
    #cv2.imwrite(new_filename, lane_line_markings)

    # Display the image
    #cv2.imshow("Image", lane_line_markings)

    # Display the window until any key is pressed
    cv2.waitKey(0)

    # Close all windows
    cv2.destroyAllWindows()

main()
```

Now that you have all the code to detect lane lines in an image, let's explain what each piece of the code does.

Isolate Pixels That Could Represent Lane Lines

The first part of the lane detection process is to apply thresholding (I'll explain what this term means in a second) to each video frame so that we can eliminate things that make it difficult to detect lane lines. By applying thresholding, we can **isolate the pixels that represent lane lines.**

Glare from the sun, shadows, car headlights, and road surface changes can all make it difficult to find lanes in a video frame or image.

NFTs and Mystery Boxes

Alternative for SMEs - Free Forever

by Customers, 2 Weeks Implementation, Accounts Integration

stranzact.com/free_erp

Before thresholding:



Image Source: Wikipedia

After thresholding:



Image Source: Wikipedia

Thresholding Steps

1. Convert the video frame from BGR (blue, green, red) color space to HLS (hue, saturation, lightness).

There are a lot of ways to represent colors in an image. If you've ever used a program like Microsoft Paint or Adobe Photoshop, you know that one way to represent a color is by using the RGB color space (**in OpenCV it is BGR instead of RGB**), where every color is a mixture of three colors, red, green, and blue. You can play around with the RGB color space here at this website.

Alternative for SMEs - Free Forever
by Customers. 2 Weeks Implementation. Accounts Integration
tranzact.com/free_erp

NFTs and Mystery Boxes

The HLS color space is better than the BGR color space for detecting image issues due to lighting, such as shadows, glare from the sun, headlights, etc. We want to eliminate all these things to make it easier to detect lane lines. For this reason, we use the HLS color space, which divides all colors into hue, saturation, and lightness values.

If you want to play around with the HLS color space, there are a lot of HLS color picker websites to choose from if you do a Google search.

2. Perform Sobel edge detection on the L (lightness) channel of the image to detect sharp discontinuities in the pixel intensities along the x and y axis of the video frame.

Sharp changes in intensity from one pixel to a neighboring pixel means that an edge is likely present. We want to detect the strongest edges in the image so that we can isolate potential lane line edges.

NFTs and Mystery Boxes

3. Perform binary thresholding on the S (saturation) channel of the video frame.

Doing this helps to eliminate dull road colors.

A high saturation value means the hue color is pure. We expect lane lines to be nice, pure colors, such as solid white and solid yellow. Both solid white and solid yellow, have high saturation channel values.

Binary thresholding generates an image that is full of 0s (black) and 255 (white) intensity values. Pixels with high saturation values (e.g. > 80 on a scale from 0 to 255) will be set to white, while everything else will be set to black.

Feel free to play around with that threshold value. I set it to 80, but you can set it to another number, and see if you get better results.

4. Perform binary thresholding on the R (red) channel of the original BGR video frame.

Alternative for SMEs - Free Forever
by Customers, 2 Weeks Implementation, Accounts Integration
stranzact.com/free_erp

To generate our binary image at this stage, pixels that have rich red channel values (e.g. > 120 on a scale from 0 to 255) will be set to white. All other pixels will be set to black.

5. Perform the bitwise AND operation to reduce noise in the image caused by shadows and variations in the road color.

Lane lines should be pure in color and have high red channel values. The bitwise AND operation reduces noise and blacks-out any pixels that don't appear to be nice, pure, solid colors (like white or yellow lane lines.)

The **get_line_markings(self, frame=None)** method in **lane.py** performs all the steps I have mentioned above.

If you uncomment this line below, you will see the output:

```
cv2.imshow("Image", lane_line_markings)
```

To see the output, you run this command from within the directory with your test image and the lane.py and edge_detection.py program.

```
python lane.py
```



For best results, play around with this line on the **lane.py** program. Move the 80 value up or down, and see what results you get.

```
_, s_binary = edge.threshold(s_channel, (80, 255))
```

Now that we know how to isolate lane lines in an image, let's continue on to the next step of the lane detection process.

Apply Perspective Transformation to Get a Bird's Eye View

We now know how to isolate lane lines in an image, but we still have some problems. Remember that one of the goals of this project was to calculate the radius of curvature of the road lane. Calculating the radius of curvature will enable us to know which direction the road is turning. But we can't do this yet at this stage due to the perspective of the camera. Let me explain.

Why We Need to Do Perspective Transformation

Alternative for SMEs - Free Forever
py Customers, 2 Weeks Implementation, Accounts Integration
stranzact.com/free_erp

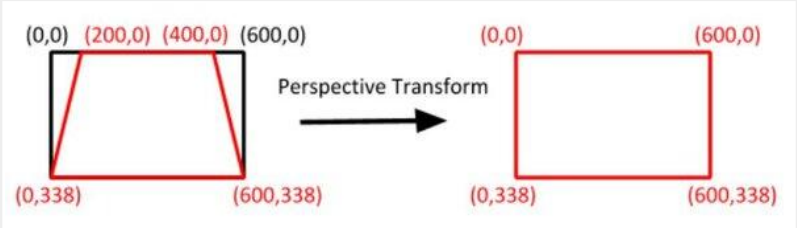
However, from the perspective of the camera mounted on a car below, the lane lines make a trapezoid-like shape. We can't properly calculate the radius of curvature of the lane because, from the camera's perspective, the lane width appears to decrease the farther away you get from the car.

In fact, way out on the horizon, the lane lines appear to converge to a point (known in computer vision jargon as **vanishing point**). You can see this effect in the image below:



The camera's perspective is therefore not an accurate representation of what is going on in the real world. We need to fix this so that we can calculate the curvature of the land and the road (which will later help us when we want to steer the car appropriately).

How Perspective Transformation Works



Fortunately, OpenCV has methods that help us perform perspective transformation (i.e. projective transformation or projective geometry). These methods **warp** the camera's perspective into a birds-eye view (i.e. aerial view) perspective.

For the first step of perspective transformation, we need to identify a region of interest (ROI). This step helps remove parts of the image we're not interested in. We are only interested in the lane segment that is immediately in front of the car.

You can run lane.py from the previous section. With the image displayed, hover your cursor over the image and find the four key corners of the trapezoid.

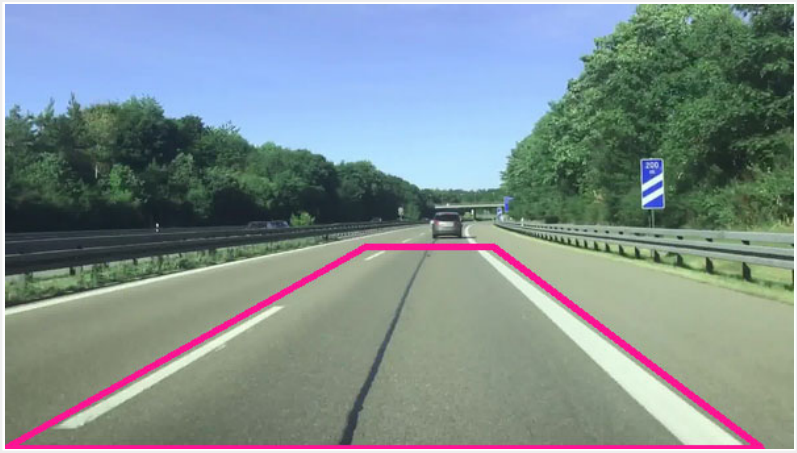
Write these corners down. These will be the roi_points (roi = region of interest) for the lane. In the code (which I'll show below), these points appear in the ___init___ constructor of the Lane class. They are stored in the **self.roi_points** variable.

Alternative for SMEs - Free Forever
py Customers, 2 Weeks Implementation, Accounts Integration
stranzact.com/free_erp

Run lane.py

```
python lane.py
```

Here is an example ROI output:



You can see that the ROI is the shape of a trapezoid, with four distinct corners.

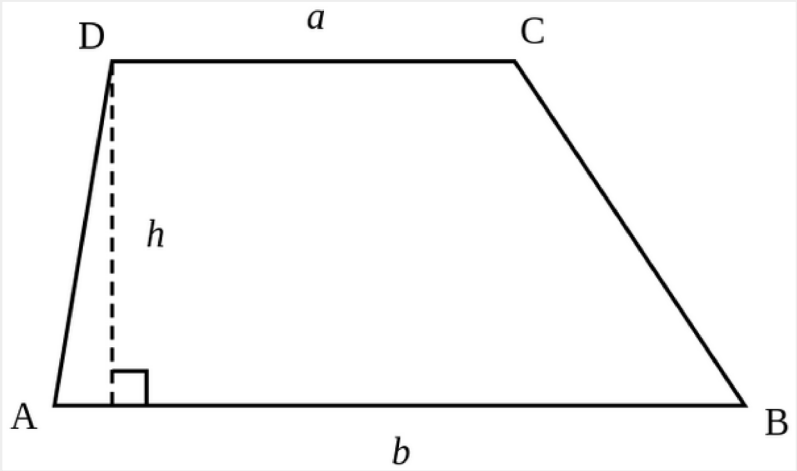


Image Source: Wikipedia

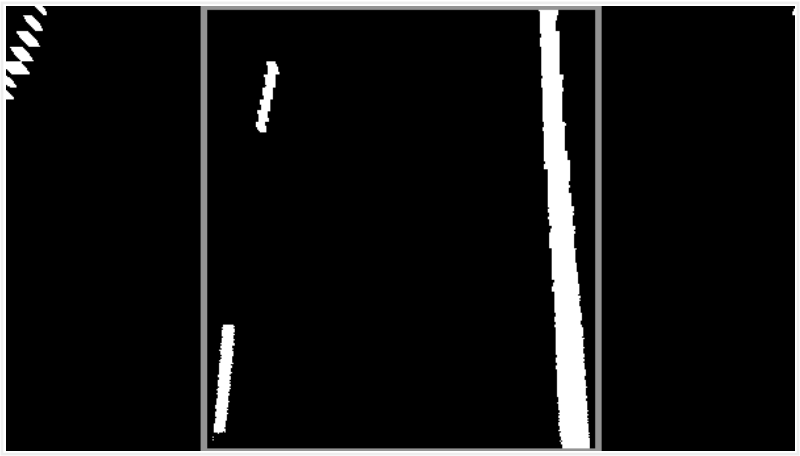
Now that we have the region of interest, we use OpenCV's **getPerspectiveTransform** and **warpPerspective** methods to transform the **trapezoid-like** perspective into a **rectangle-like** perspective.

Change the parameter value in this line of code in lane.py from False to True.

```
warped_frame = lane_obj.perspective_transform(plot=True)
```

Here is an example of an image after this process. You can see how the perspective is now from a birds-eye view. The ROI lines are now parallel to the sides of the image, making it easier to calculate the curvature of the road and the lane.

Alternative for SMEs - Free Forever
py Customers 2 Weeks Implementation Accounts Integration
stranzact.com/free_erp



Identify Lane Line Pixels

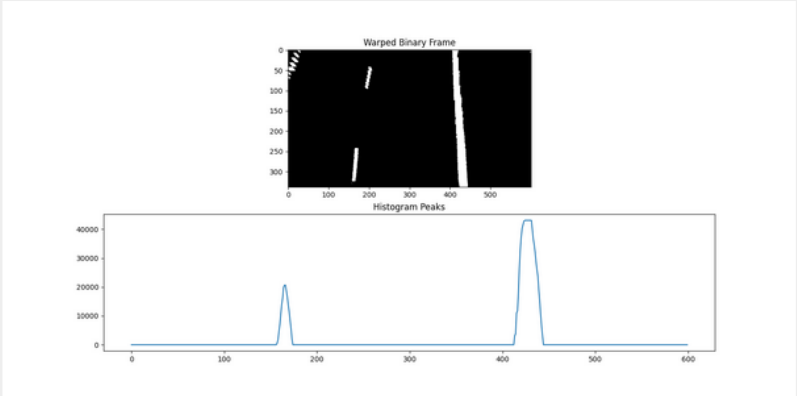
We now need to identify the pixels on the warped image that make up lane lines. Looking at the warped image, we can see that white pixels represent pieces of the lane lines.

We start lane line pixel detection by generating a histogram to locate areas of the image that have high concentrations of white pixels.

Ideally, when we draw the histogram, we will have two peaks. There will be a left peak and a right peak, corresponding to the left lane line and the right lane line, respectively.

In lane.py, make sure to change the parameter value in this line of code (inside the main() method) from False to True so that the histogram will display.

```
histogram = lane_obj.calculate_histogram(plot=True)
```



Set Sliding Windows for White Pixel Detection

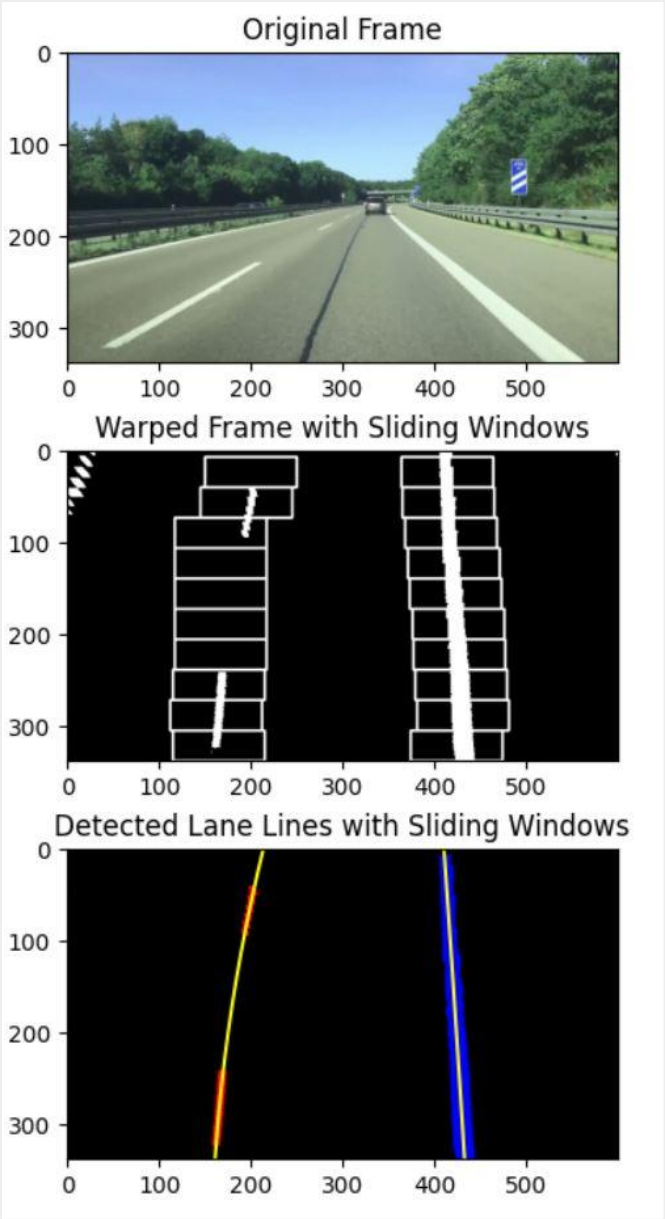
The next step is to use a sliding window technique where we start at the bottom of the image and scan all the way to the top of the image. Each time we search within a sliding window, we add potential lane line pixels to a list. If we have enough lane line

Alternative for SMEs - Free Forever
py Customers, 2 Weeks Implementation, Accounts Integration
stranzact.com/free_erp

In this line of code, change the value from False to True.

```
left_fit, right_fit = lane_obj.get_lane_line_indices_sliding_windows(
     plot=True)
```

Here is the output:

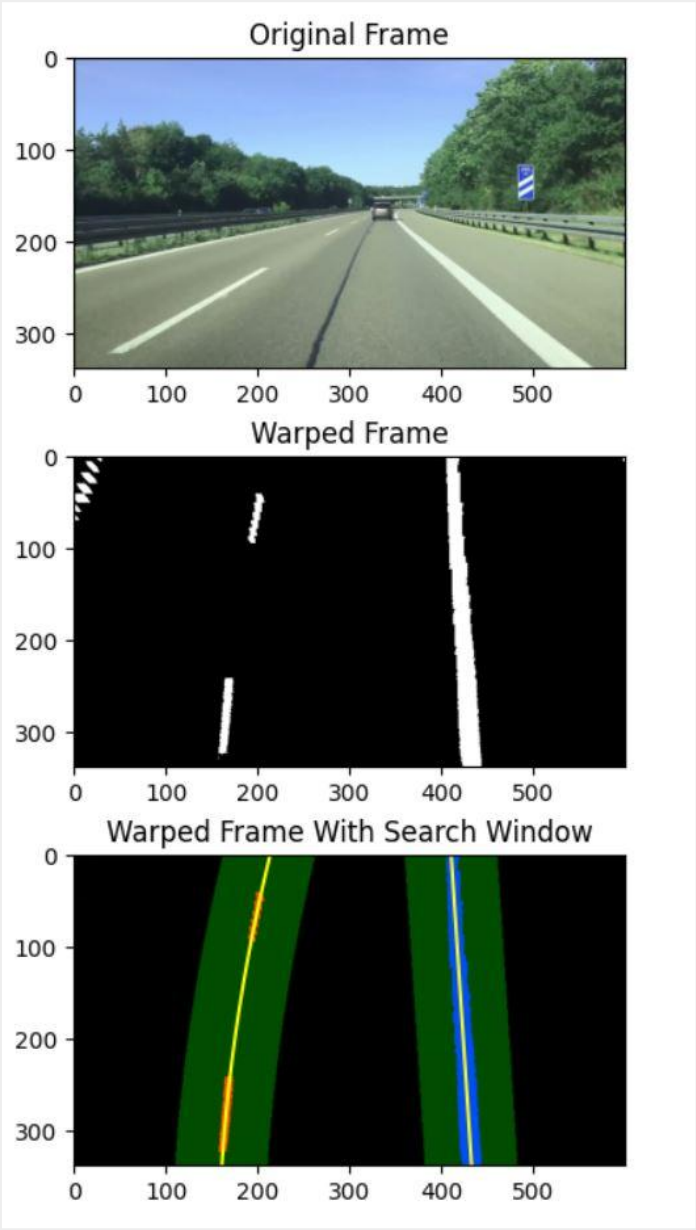


Fill in the Lane Line

Now let's fill in the lane line. Change the parameter value on this line from False to True.

```
lane_obj.get_lane_line_previous_window(left_fit, right_fit, plot=True)
```

Alternative for SMEs - Free Forever
py Customers, 2 Weeks Implementation, Accounts Integration
stranzact.com/free_erp



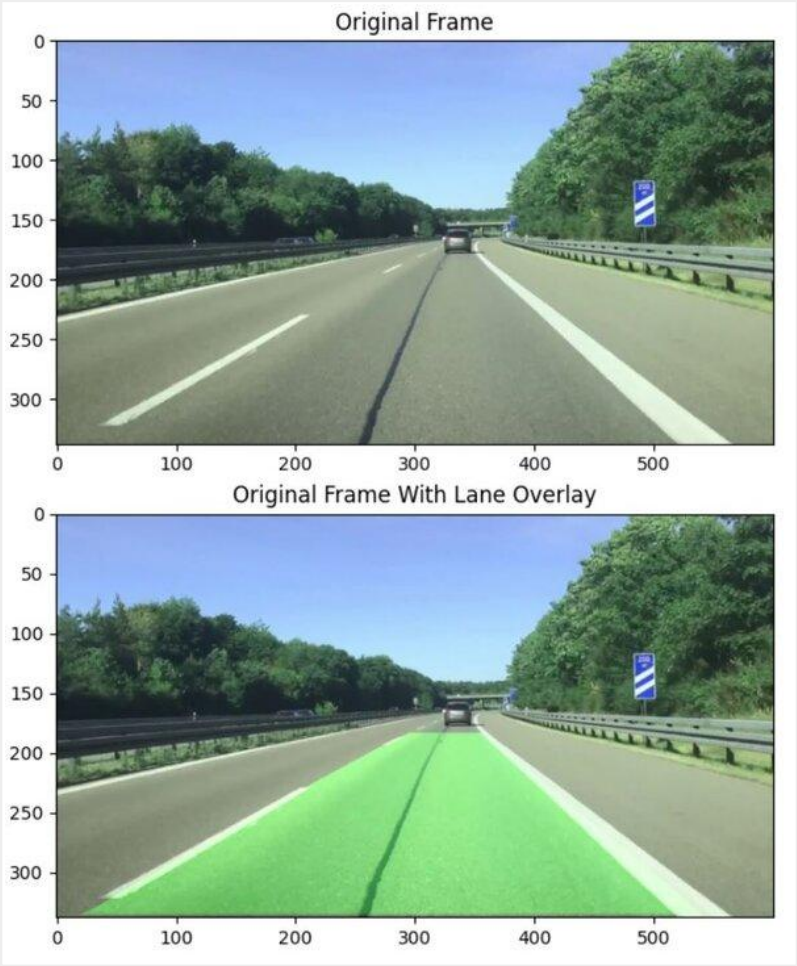
Overlay Lane Lines on Original Image

Now that we've identified the lane lines, we need to overlay that information on the original image.

Change the parameter on this line form False to True and run lane.py.

```
frame_with_lane_lines = lane_obj.overlay_lane_lines(plot=True)
```

Alternative for SMEs - Free Forever
by Customers. 2 Weeks Implementation. Accounts Integration.
https://tranzact.com/free_erp



Calculate Lane Line Curvature

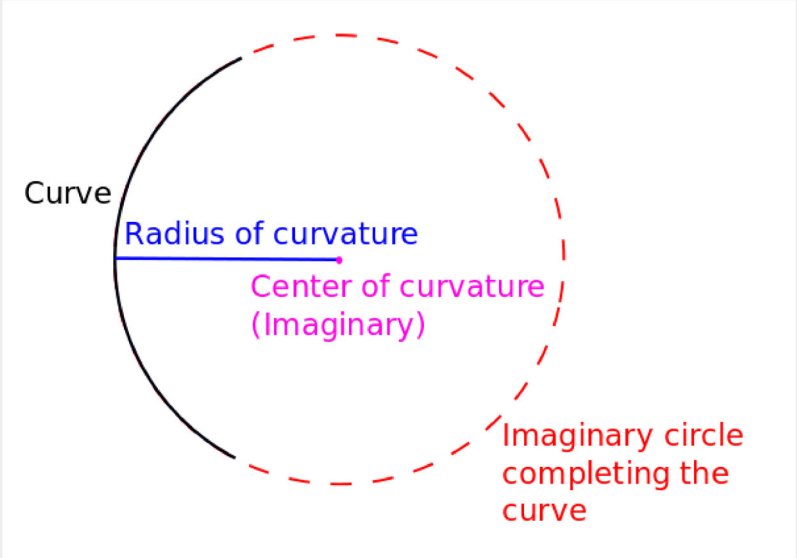


Image Source: Wikipedia

Alternative for SM^Es - Free Forever
py Customers, 2 Weeks Implementation, Accounts Integration
stranzact.com/free_erp

Here is the output. You can see the radius of curvature from the left and right lane lines:

```
40.31949794887235 m 3772.427228241085 m
```

Calculate the Center Offset

Now we need to calculate how far the center of the car is from the middle of the lane (i.e. the "center offset).

On the following line, change the parameter value from False to True.

```
lane_obj.calculate_car_position(print_to_terminal=True)
```

Run the program.

```
python lane.py
```

Here is the output. You can see the center offset in centimeters:

```
1.435914726167449cm
```

Display Final Image

Now we will display the final image with the curvature and offset annotations as well as the highlighted lane.

In lane.py, change this line of code from False to True:

```
frame_with_lane_lines2 = lane_obj.display_curvature_offset(
     frame=frame_with_lane_lines, plot=True)
```

Run lane.py.

```
python lane.py
```

Here is the output:



Alternative for SMEs - Free Forever
py Customers 2 Weeks Implementation Accounts Integration
stranzact.com/free_erp

Detect Lane Lines in a Video

Now that we know how to detect lane lines in an image, let's see how to detect lane lines in a video stream.

All we need to do is make some minor changes to the main method in **lane.py** to accommodate video frames as opposed to images.

Here is the code for lane.py. It takes a video in mp4 format as input and outputs an annotated image with the lanes.

Troubleshooting

For best results, play around with this line on the lane.py program. Move the 80 value up or down, and see what results you get.

```
_, s_binary = edge.threshold(s_channel, (80, 255))
```

If you run the code on different videos, you may see a warning that says **"RankWarning: Polyfit may be poorly conditioned"**. If you see this warning, try playing around with the **dimensions of the region of interest** as well as the **thresholds**.

You can also play with the length of the moving averages. I used a 10-frame moving average, but you can try another value like 5 or 25:

```
if len(prev_left_fit2) > 10:
```

Using an exponential moving average instead of a simple moving average might yield better results as well.

That's it for lane line detection. Keep building!

automaticaddison / April 12, 2021 / Computer Vision, Robotics / ground, opencv, python

Alternative for SMEs - Free Forever
py Customers 2 Weeks Implementation Accounts Integration
stranzact.com/free_erp